DAY 3

Python Data Types, Operators, Mutable & Immutable Objects

Learning goals: Understand Python's built-in data types, use the main operator groups, and distinguish mutable objects from immutable objects.

1. Python Data Types

A **data type** tells Python what kind of value an object represents. You can check a value's type with `type()`.

Order	Data type	What it stores	Example
1	int	Whole numbers	`age = 20`
2	float	Numbers with decimal points	`price = 99.5`
3	complex	Complex numbers	`z = 2 + 3j`
4	bool	True or False	`is_valid = True`
5	str	Text / characters	`name = 'Raghu'`
6	list	Ordered, changeable collection; duplicates allowed	`items = [10, 20, 10]`
7	tuple	Ordered, unchangeable collection; duplicates allowed	`point = (10, 20)`
8	range	A sequence of integers, often used in loops	`r = range(5)`
9	dict	Key-value pairs; keys are unique	`student = {'name': 'Raghu'}`
10	set	Unordered collection of unique items	`nums = {1, 2, 3}`
11	frozenset	Immutable set	`fs = frozenset({1, 2})`
12	NoneType	Absence of a value	`result = None`
13	bytes	Immutable sequence of bytes	`data = b'ABC'`
14	bytearray	Mutable sequence of bytes	`data = bytearray(b'ABC')`
15	memoryview	A view of binary data without copying it	`view = memoryview(b'ABC')`

Example: checking data types

```
age = 20
price = 99.5
name = "Raghu"
is_student = True

print(type(age))        # <class 'int'>
print(type(price))      # <class 'float'>
print(type(name))       # <class 'str'>
print(type(is_student)) # <class 'bool'>
```

Note: Python does not require you to declare a variable's type. The type is determined by the value assigned to it.

2. Python Operators

Operators are symbols or keywords that perform operations on values and variables.

A. Arithmetic operators

Operator	Meaning	Example	Result
+	Addition	10 + 3	13
-	Subtraction	10 - 3	7
*	Multiplication	10 * 3	30
/	Division	10 / 3	3.333...
//	Floor division	10 // 3	3
%	Remainder (modulus)	10 % 3	1
**	Power	2 ** 3	8

B. Comparison operators

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	5 > 3	True
<	Less than	5 < 3	False
>=	Greater than or equal to	5 >= 5	True
<=	Less than or equal to	3 <= 5	True

C. Assignment operators

Operator	Example	Equivalent meaning
=	x = 10	Assign 10 to x
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 2	x = x * 2
/=	x /= 2	x = x / 2
//=	x //= 2	x = x // 2
%=	x %= 3	x = x % 3
**=	x **= 2	x = x ** 2

D. Logical operators

Operator	Meaning	Example	Result
and	True if both conditions are true	True and False	False
or	True if at least one condition is true	True or False	True
not	Reverses a Boolean result	not True	False

E. Membership operators

```
fruits = ["apple", "banana", "mango"]
print("apple" in fruits)       # True
print("grapes" not in fruits)  # True
```

`in` checks whether an item is present; `not in` checks whether it is absent.

F. Identity operators

```
a = None
b = None
print(a is b)      # True
print(a is not b)  # False
```

== compares values; `is` checks whether two references point to the same object. Use `is None` when checking for None.

G. Bitwise operators

Operator	Meaning	Example	Result
&	Bitwise AND	5 & 3	1
|	Bitwise OR	5 | 3	7
^	Bitwise XOR	5 ^ 3	6
~	Bitwise NOT	~5	-6
<<	Left shift	5 << 1	10
>>	Right shift	5 >> 1	2

Example: using several operators

```
a = 10
b = 3

print(a + b)                # 13
print(a % b)                # 1
print(a > b)                # True
print(a > 5 and b < 5)      # True
```

3. Mutable and Immutable Objects

Mutable: an object can be changed after it is created. **Immutable:** an object cannot be changed after it is created. Reassigning a variable is not the same as changing the original object.

Mutable objects	Immutable objects
list — `[1, 2, 3]`	int — `10`
dict — `{'a': 1}`	float — `2.5`
set — `{1, 2, 3}`	bool — `True`
bytearray — `bytearray(b'ABC')`	str — `'hello'`
	tuple — `(1, 2, 3)`
	frozenset, bytes, range, NoneType, complex

Example 1: mutable list

```
numbers = [10, 20, 30]
numbers[1] = 100
print(numbers)  # [10, 100, 30]
```

Example 2: immutable string

```
word = "Python"
# word[0] = "J"  # TypeError: strings cannot be changed in place
word = "Jython"   # This reassigns the variable to a new string
print(word)       # Jython
```

Example 3: immutable tuple

```
numbers = (10, 20, 30)
print(numbers)  # (10, 20, 30)

# numbers[0] = 100  # TypeError: tuple items cannot be reassigned
```

Example 4: mutable dictionary

```
student = {"name": "Raghu", "age": 20}
student["age"] = 21
print(student)  # {'name': 'Raghu', 'age': 21}
```

Remember: a tuple is immutable, but it can contain a mutable object such as a list. The tuple's element references cannot be reassigned, though the nested list itself may change.

4. Quick Revision

Question	Answer
Which type stores whole numbers?	int
Which type stores decimal numbers?	float
Which collection is ordered and mutable?	list
Which collection is ordered and immutable?	tuple
Which type stores key-value pairs?	dict
Which operator returns a remainder?	%
Which operator checks equality?	==
Which keyword checks membership?	in
Which keyword checks object identity?	is

Practice: Write a program that stores your name, age, and marks, prints each value's type, and uses arithmetic and comparison operators.